

Technical report — Apertus QA: grounded Spanish answers over official statistics

- **Track:** Track 2B — Apertus QA (own project)
- **Event:** Online
- **Team / entrant:** Vento Labs
- **Demo:** https://github.com/ventolabs-hq/apertus-qa/blob/38b937c3664e03e9a021dbd8881b463cefc3a04e/track_2b/demo/apertus-qa-demo.mp4 (≤ 2 min, screen capture + captions)
- **Code:** [@38b937c3664e03e9a021dbd8881b463cefc3a04e](https://github.com/ventolabs-hq/apertus-qa)

Status. Numbers marked *STUB* come from replayed outputs **written by the build agent, not by Apertus**, or from the no-model baseline. Numbers marked **Apertus** come from real calls to `swiss-ai/Apertus-v1.5-70B` (and, for comparison, `swiss-ai/Apertus-v1.5-8B`) on the CSCS inference endpoint (4 Oct 2026; raw outputs in `docs/eval/eval_*_record.*` and `eval_*_real_8b.*`, summary in `docs/eval/REAL_RUN_2026-10-04.md`).

1. Summary

General-purpose chatbots answer statistical questions fluently but often with unsourced, stale or wrong figures. For a newsroom, public office or school that is worse than no answer. **Apertus QA** answers Spanish questions over a local, licence-checked snapshot of official Argentine open data (47 series and tables from INDEC and other agencies, CC BY 4.0). Apertus does the language work: it **plans** the query (series, operation, period) and **phrases** the answer. It never produces or sees a number. Deterministic code computes every figure and attaches the citation. The system says when the data cannot answer, and refuses forecasts and financial advice. Everything except the model call runs with `docker run --network none`, and the model can be a self-hosted Apertus on the same isolated network. *STUB/baseline*: main set 31/31, held-out paraphrases 8/13, 0 grounding violations, 5/5 INDEC cross-checks. **Apertus v1.5-70B (real, CSCS)**: main 30/31, held-out paraphrases **12/13** (baseline 8/13), 0 grounding violations, 0 false refusals, 5/5 INDEC cross-checks; median 2.5 s per question. Apertus v1.5-8B with the same prompts: 22/31 and 9/13.

2. Architecture

```
question → P1 router (Apertus) → plan JSON → validate_plan: catalog ids, period, op,
    |                                     amount must appear in the question
    |→ invalid / timeout / no model → rules router
    |
ops.run: 13 deterministic operations over data/snapshot (sha256 + licence checked at start)
    |                                     ↓ figures · placeholder slots · citations
P2 phrasing (Apertus; placeholder NAMES + hints only) → validate_phrase: no digits or
    |                                     number words, known placeholders only
    |→ invalid / timeout / no model → per-operation template
    |
render → grounding re-check → answer + citation + figures + trace (Web UI · JSON API · CLI)
```

One container, one stateless process: Python 3.12 **standard library only** (no pip install, no database). The ≈0.3 MB snapshot ships in the image. The Spanish UI is one HTML file with no external assets (a test asserts it). Modules (`src/apertus_qa/`): `catalog`, `ops` (value, latest, change, monthly, year-on-year, max, min, annual total, ranking, inflation adjustment, comparison, table top-n/value), `gateway`, `providers` (OpenAI-compatible, stub replay, recorder), `prompts`, `validate`, `rules`, `pipeline`, `server`. **LLM gateway** (ported from

the team's *Ciclo* prototype, §9): never throws; per-minute/per-day budgets, timeout, circuit breaker (transport failures only), tolerant JSON parsing, schema + semantic validation, deterministic fallback with a machine-readable reason (`network_error`, `http_error`, `bad_response`, `budget_exhausted`, `breaker_open`, validator rejection) shown in the trace and the UI.

Grounding guarantees (each covered by tests):

#	Guarantee	Mechanism
G1	Numbers come only from code	<code>ops.py</code> computes from the verified snapshot
G2	The phrasing model never sees a figure	Gets placeholder names and hints (<code>direccion: subió</code>); a test with a fake endpoint asserts the figure is absent from the request
G3	Model text cannot carry numbers	Any digit or number word → rejected → template
G4	The router cannot invent inputs	Series must exist in the catalog; periods well-formed; an amount to adjust must appear in the question
G5	Final text is re-checked	Every numeric token must equal a computed slot value
G6	Citations are written by code	Publisher, series id, period, licence, portal

Residual failure mode: a valid but *wrong* plan gives a real, cited figure for a slightly different question (§6). The UI shows series and period next to every answer so users can spot it.

Target architecture (mandatory): b) Air-gapped — also a) on-premise and c) sovereign Swiss cloud

Stage	External network	How
Build	Base image only (<code>python:3.12-slim</code>)	All RUN steps run under <code>docker build --network none</code> . Isolated sites: <code>make image-save</code> → <code>docker load</code> .
Data	None	Snapshot inside the image; sha256 + licence checked at start; refreshed outside the site as a new image.
Runtime: UI, API, computation, citations	None	Proven with <code>--network none</code> (below).
Runtime: model	Only <code>LLM_BASE_URL</code>	CSCS-hosted Apertus (Switzerland) or self-hosted (vLLM/Ollama/llama.cpp, OpenAI-style API, key optional) in the same isolated network.
Model unreachable	None	Rules + templates: answers stay correct and cited; the UI shows the mode.

Air-gap proof (`make offline-proof`; full transcript in `docs/OFFLINE.md`). Under `--network none` only `lo` exists; connections to `1.1.1.1:443` / `8.8.8.8:53` fail (Network is unreachable) and DNS fails; the server answers `/health` and `/api/ask` inside the container; 41 tests and all four STUB/off eval runs pass with unchanged scores; in *real* mode the trace shows fallback (`network_error`) and the answer is still correct and cited — the model call is the only network dependency. `docs/deploy/docker-compose.airgap.yml` sketches app + vLLM with local Apertus weights on an `internal: true` network (not run: no GPU).

3. Use of Apertus

Aspect	Setup
Model	<code>swiss-ai/Apertus-v1.5-70B</code> : the largest non-"thinking" Apertus model listed by the CSCS endpoint's <code>/models</code> (which also lists <code>v1.5-8B</code> , the 2509 Instruct 8B/70B and <code>v1.5 thinking</code> variants)
Usage	Inference in two roles: P1 tool-use-style planning (one JSON plan over a 47-entry catalog) and P2 constrained generation (Spanish phrasing with placeholders only)
Serving	CSCS endpoint or any OpenAI-style server via <code>LLM_BASE_URL</code> , <code>LLM_NAME</code> , <code>LLM_API_KEY</code> ; <code>/chat/completions</code> , temperature 0; optional <code>response_format: json_object</code>

Aspect	Setup
P1 prompt	Spanish instructions + compact catalog (ids, titles, units, frequency, coverage; no values ; ≈4k tokens) → {action, op, series, period, from, to, amount, n, ...} or {action: "refuse", reason} (out of scope / forecast / advice / other). Out-of-coverage periods pass through so code can explain coverage.
P2 prompt	Question, operation, placeholder descriptions, direction hints → {"texto": ...}, one or two sentences
Record/replay	LLM_MODE=record saves real outputs to a separate file; LLM_MODE=replay replays them with no key or network (labelled REPLAY; the STUB never loads it). The replay reproduces the 70B run row by row (30/31, 12/13), also under --network none

Observed behaviour (real runs, 44 questions, 78 calls). Every output parsed as JSON (0 bad_json); 0 HTTP errors, 0 timeouts, 0 retries. *Router*: plan used for 24/31 main and 13/13 held-out questions; the 7 main rejections went to the rules router (period format 2, unknown/ill-formed series 2, table/operation mismatch 1, change operation on a series that is already a % change 2). *Phrasing*: text used for 11/23 main and 5/11 held-out answers; rejections (→ template) were digits (main 10, held-out 4; on the main set mostly years copied from the question), number words (1 / 2) and a missing placeholder (1 / 0). *Latency* per question (router + phrasing + computation): main p50 2.5 s, p95 3.5 s, max 3.6 s; held-out p50 2.4 s, p95 3.0 s, max 3.0 s. *Tokens*: 171,117 (main, 54 calls) and 72,100 (held-out, 24 calls). *Prompt iterations*, on the main set only: 19 → 21 → 29 → 29 → 30/31. The first real run refused 7 questions about 2025–26 as "forecasts"; giving the router the snapshot date plus six few-shot plans fixed that; a validator now sends "% change of a % series" plans to the rules router. A phrasing-prompt pass (no copied dates, % already in {variacion}) lowered phrasing use on the main set (11 → 8 of 23, score unchanged) and was reverted.

No other model is used at runtime or in evaluation (scoring is deterministic code, no LLM judge).

4. Data

Domain	Content	Publisher
Prices, wages	CPI national (monthly, Dec 2016 → Aug 2026), Dec–Dec inflation; registered wage index	INDEC
Population, births	Population 2004–2025; live births (to 2022, declared)	INDEC; DEIS / Min. Salud
Exchange rate, trade	Nominal annual exchange rate; exports, total + 4 categories	INDEC / Min. Economía
Tourism	Resident tourist trips by destination and mode (27 series)	Subsecretaría de Turismo
Energy	Installed solar / wind / total capacity; crude oil production	CAMMESA; Secretaría de Energía
Transport, road safety	Subway passengers; 2025 flights by airport and route (counts only); road deaths	INDEC; ANAC; SNIC

All data are **third-party open data under CC BY 4.0** from datos.gob.ar / apis.datos.gob.ar, fetched at build time only, respecting robots.txt and its crawl delay. We redistribute them under their original licence **with attribution** (data/README.md, docs/DATA.md, and a citation on every answer); we do not relicense them or publish them as our own dataset. Transformations: time-window trimming; flight and road-death microdata reduced to aggregate counts; no personal data. Per-file sha256 in catalog.json.

5. Evaluation

Sets. *Main*: 31 Spanish questions — 23 answerable, 8 to refuse (missing period, forecast, out of scope, advice, prompt injection). *Held-out*: 13 paraphrases (colloquial, elliptical) written separately and not used to build the rules router or prompts. **Ground truth** is

computed from the snapshot by independent code (`src/eval/make_expected.py`, which does not import the app); five CPI answers are also checked against INDEC's published figures. **Metrics:** decision (answer vs refuse), value match, citation (series + period), grounding violations (numbers not traceable to a computed figure; target 0), false refusals, fallback rates by reason.

Setup	Set	Overall	Answers	Refusals	False refusals	Grounding viol.
No-model baseline (rules + templates)	main	31/31	23/23	8/8	0	0
STUB replay (agent-written, not Apertus)	main	31/31	23/23	8/8	0	0
No-model baseline	held-out	8/13	6/11	2/2	4	0
Apertus v1.5-70B (real)	main	30/31	23/23	7/8	0	0
Apertus v1.5-70B (real)	held-out	12/13	10/11	2/2	0	0
Apertus v1.5-8B (real, same prompts)	main	22/31	15/23	7/8	8	0
Apertus v1.5-8B (real, same prompts)	held-out	9/13	7/11	2/2	4	0

Baseline and STUB rows reproduced in the Docker image with `--network none`; Apertus rows run from the same code on the host (`LLM_MODE=real make eval-local`, smoke-tested in the image); INDEC cross-checks 5/5 in the final main runs. **Reading.** The rules router was written while looking at the main set, so its 31/31 is not blind; the held-out set is the fair comparison. Claim under test: *Apertus raises held-out coverage above the baseline's 8/13 while keeping grounding violations at 0.* STUB rows only show that pipeline, validators and fallbacks work; its four adversarial outputs (unknown series id, markdown-fenced plan, invented figure, banned word "récord") are all handled as designed.

Error analysis (Apertus). The 4 held-out gains over the baseline are paraphrases the rules router misses: synonyms ("gente" for population, "ventas al exterior" for exports, "en avión" for air trips) and a salary amount to adjust by CPI that the rules sent to the wage index. All final-run held-out plans passed validation. Two misses, neither with an invented number: (1) main "¿Cuántos nacimientos hubo en 2024?" was refused as out of scope instead of planned on the births series (which ends in 2022), so the user is not told the coverage limit; (2) held-out "¿Qué tan cara se puso la vida en 2024, punta a punta?" was planned as the Nov → Dec 2024 change (2.7 %) instead of the Dec–Dec annual change: a valid but wrong plan, shown with its series and period, i.e. the residual failure mode in §2. Plan errors caught by validators: 7/31 main, 0/13 held-out; wrong plans that passed: 1 (held-out). The endpoint is not fully deterministic at temperature 0: in one earlier run with the same prompts, miss (2) was a refusal instead; the score was 12/13 in all held-out runs. **8B** (p50 1.8 s main / 2.0 s held-out; p95 2.3 s / 4.1 s) mostly over-refuses (recent CPI as "forecast"; subway, tourism and oil questions as out of scope). On one must-refuse question ("¿Cuántos pasajeros transportaron las aerolíneas en 2025?") it answered with the real 2025 subway + premetro total, correctly cited, but worded as airline passengers: it passes the numeric grounding check (G5 verifies numbers, not labels), so we ship 70B and list label checks as future work.

6. Limitations

Fixed coverage (47 entries; no unemployment, poverty or GDP yet; CPI to Aug 2026). A valid but wrong plan can answer a slightly different question (1/13 held-out for the baseline); plan-question consistency checks are planned. Spanish only, single turn. Real-model results come from 44 questions, with prompts iterated on the 70B main set; grounding checks numbers, not the wording around them (see the 8B case); self-hosted Apertus untested (no GPU); no load or accessibility testing.

7. Reproducibility

From `track_2b/`: `make run` (UI at `http://localhost:8080`), `make test`, `make eval`, `make eval-heldout`, `make offline-proof`, `make report` (this PDF). STUB and off modes are deterministic: `docs/eval/` regenerates identically (apart from a timing field). Verified on Docker Engine 29.8.2, Linux x86-64, CPU only. Without Docker: `make run-local / test-local / eval-local` (Python ≥ 3.10). Real runs: set `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`, then `LLM_MODE=record make eval`; without a key, `LLM_MODE=replay make eval` replays the recorded 70B run offline. Code commit: `38b937c3664e03e9a021dbd8881b463cefc3a04e` (the commit after it only fills in these links and rebuilds this PDF).

8. Next steps

A larger fresh held-out set; plan-question and series-label consistency checks; more series (labour market, GDP, provinces); a second statistics office (e.g. BFS / `opendata.swiss`) through the same catalog format; a measured self-hosted deployment on an isolated GPU node.

9. Disclosures

Topic	Statement
AI-assisted development	Built with closed-weights (proprietary) AI coding agents , directed and reviewed by the entrant. They wrote the code, tests, docs and this report. They are not part of the submitted system, are not called at runtime, and are not used as judges. The only runtime model is Apertus. (The rules mention open-weights models supporting development; ours were closed-weights, so we state their role explicitly.)
STUB outputs	Until real runs are recorded, the STUB replays outputs written by the build agent , labelled STUB in the UI, eval reports and this report.
Component reuse	LLM gateway ported (JavaScript $\rightarrow$ Python) from the team's Ciclo prototype, written 2 Oct 2026; one change: invalid content no longer trips the breaker. Container/env-var/health pattern adapted from Ciclo.
Data reuse	Raw downloads, licence evidence and aggregation definitions from the team's open-data pipeline (1–2 Oct 2026), re-verified at build time.
Template	HackApertus/project-template @ 7f23822 (Apache-2.0). Details: <code>docs/DISCLOSURE.md</code> .
Data licence	Third-party CC BY 4.0 with attribution (§4).

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. Code: Apache-2.0 (LICENSE). Data in `data/`: third-party, CC BY 4.0 (original publishers).

References

Apertus v1.5 model card (`huggingface.co/swiss-ai/Apertus-v1.5-8B`) · Series de Tiempo API, `datos.gob.ar` (`datosgobar.github.io/series-tiempo-ar-api`) · INDEC, Índice de precios al consumidor (`indec.gob.ar`) · Hack Apertus project template (`github.com/HackApertus/project-template @ 7f23822`)